



Algorithmen und Datenstrukturen

Leon Johann Brettin



Algorithmen und Datenstrukturen

9.1 Einschub Comparator



Ordnungsrelation

- Eine Relation, die reflexiv, antisymmetrisch und transitiv ist, heißt **Ordnungsrelation**.
 - Zwei Elemente aus A heißen **vergleichbar** bezüglich R , wenn $x \leq_R y$ oder $y \leq_R x$ gilt.
 - Eine Ordnungsrelation R in A heißt **total**, wenn jedes Element aus A mit jedem vergleichbar ist.
 - Andernfalls heißt R **partiell**.
 - Bei einer totalen Ordnungsrelation R kann man sich die Elemente aus A in einer bestimmten Reihenfolge **linear angeordnet** denken.
 - Also kann man sie sortieren!



Interface `java.util.Comparator<T>`

- A comparison function, which imposes a **total ordering** on some collection of objects. Comparators can be **passed to a sort** method (such as `Collections.sort` or `Arrays.sort`) to allow precise control over the sort order. Comparators can also be used to control the order of certain data structures (such as sorted sets or sorted maps), or to provide an ordering for collections of objects that don't have a natural ordering.
- The ordering imposed by a comparator `c` on a set of elements `S` is said to be **consistent with equals** if and only if `c.compare(e1, e2) == 0` has the same boolean value as `e1.equals(e2)` for every `e1` and `e2` in `S`.
- `int compare(T o1, T o2)` - compares its two arguments for order.



Eigenen Comparator definieren

```
public class Punkt {  
    public int x,y;  
}
```



```
import java.util.Comparator;
```



```
public class XVergleicher implements Comparator<Punkt> {  
    // liefert Wert < 0, wenn p1 < p2 ist.  
    // liefert 0, wenn p1 == p2 ist.  
    // liefert Wert > 0, wenn p1 > p2 ist.  
    public int compare(Punkt p1, Punkt p2){  
        int dx = p1.x-p2.x;  
        return dx;  
    }  
}
```

- Problem: konsistent mit p1.equals(p2)?



Eigenen Comparator definieren

```
import java.util.Comparator;

public class XYVergleicher implements Comparator<Punkt> {
    // liefert Wert < 0, wenn p1 < p2 ist.
    // liefert 0, wenn p1 == p2 ist.
    // liefert Wert > 0, wenn p1 > p2 ist.
    public int compare(Punkt p1, Punkt p2) {
        int dx = p1.x - p2.x;
        if (dx != 0)
            return dx;
        int dy = p1.y - p2.y;
        return dy;
    }
}
```





java.lang.Comparable

- Wenn es nur eine Möglichkeit gibt, die Objekt zu ordnen, dann kann das Objekt selbst den Vergleich implementieren:
- `public interface Comparable<T>`
 - This interface imposes a total ordering on the objects of each class that implements it. This ordering is referred to as the class's natural ordering, and the class's `compareTo` method is referred to as its natural comparison method.
 - `int compareTo(T o)` - compares this object with the specified object for order.



java.lang.Comparable - 2

- The implementor must ensure
 - $\text{sgn}(x.\text{compareTo}(y)) == -\text{sgn}(y.\text{compareTo}(x))$ for all x and y .
 - This implies that $x.\text{compareTo}(y)$ must throw an exception iff $y.\text{compareTo}(x)$ throws an exception.
- Must also ensure that the relation is transitive:
 - $(x.\text{compareTo}(y) > 0 \ \&\& \ y.\text{compareTo}(z) > 0)$ implies $x.\text{compareTo}(z) > 0$.
- Must ensure that
 - $x.\text{compareTo}(y) == 0$ implies that $\text{sgn}(x.\text{compareTo}(z)) == \text{sgn}(y.\text{compareTo}(z))$, for all z .
- It is strongly recommended, but not strictly required that
 - $(x.\text{compareTo}(y) == 0) == (x.\text{equals}(y))$.